accounts. This simple testing is all right for prototyping. For more thorough testing, you need automated test scripts. In chapter 10, you'll learn about automatic testing with test scripts.

6.2.4 Accessing on-chain data using the web3 API

There are two significant differences between the testing you've done here and what you did in chapter 5: in this chapter, you have a web UI, and you are accessing data logged for the events emitted. You explored the web UI in section 6.2.3. How about the event log data? How is it delivered as a notification to the user (figure 6.7)? Figure 6.9 shows the steps involved in processing on-chain logged events.

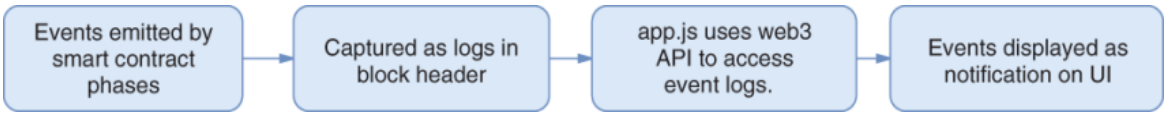


Figure 6.9 Processing of event logs using web3 API

Let's explore the app.js code for the new version of the blind auction (BlindAuctionV2-Dapp) and identify the snippets that access the logged events and notify the users. Of them, the first one shows how you access an event without parameters, and the second shows how you access an event with parameters. These examples show how you write code accessing event logs.

The following snippet is inside the `handlePhase()` function of app.js. The success of the function call execution is recorded in the blockchain receipt data structure and is checked by verifying that `status == 1`. If this statement is true, you can extract the indexed logs of the event(s) emitted as `logs[0]`, `logs[1]`, and so on. This snippet shows how the `BiddingStarted` and